# Composable Reasoning Patterns for Autonomous Red Teaming: A Multi-Agent Orchestration Framework with HITL Safety Governance

Nishan Paul*†, John Doe*, and Jane Smith‡
*Research & Development, Joblio Security Labs, Dhaka, Bangladesh
†Department of CSE, University of Dhaka, Dhaka, Bangladesh
‡Open Source Security Collective, San Francisco, USA
{nishan.paul, j.doe}@joblio.ai, smith@agentic-vapt.ai

***Abstract*—The deployment of Large Language Models (LLMs) for autonomous security operations represents a transformative shift in vulnerability assessment and penetration testing (VAPT). However, standard agentic workflows often suffer from reasoning fragility in long-horizon tasks, high operational costs in multi-step attack chains, and a lack of formal safety governance. We present CMatrix, a multi-agent orchestration framework that addresses these challenges through a composite reasoning suite. CMatrix integrates Tree of Thoughts (ToT) for strategic lookahead, ReWOO for dependency-aware planning, and a novel executable reflection mechanism that converts verbal critique into structured tool re-execution. Furthermore, we incorporate a risk-aware Human-in-the-Loop (HITL) safety gate directly into the stateful orchestration loop to govern high-risk operations. Through an extensive evaluation across 80 security reasoning tasks, we demonstrate that CMatrix achieves a 97.4% success rate while reducing token consumption by 2.42× compared to standard ReAct-based agents. Our results prove that the composition of advanced reasoning patterns significantly enhances the thoroughness and efficiency of autonomous red teaming in complex enterprise environments.**

***Index Terms*—Agentic Workflows, LLM Orchestration, Multi-Agent Systems, Autonomous VAPT, HITL Safety, Tree of Thoughts, ReWOO.**

## I. Introduction

The increasing complexity of modern cloud-native infrastructures has outpaced the capabilities of traditional static vulnerability scanners. While these tools excel at identifying known software flaws through signature matching, they are fundamentally limited in their ability to simulate "hands-on-keyboard" adversaries who chain minor misconfigurations into multi-stage attack vectors. The emergence of Large Language Models (LLMs) offers a potential solution, enabling autonomous agents to interpret unstructured tool outputs and generate adaptive strategies in real-time.

### I.1. The Challenge of Long-Horizon Security Reasoning

Despite their potential, current LLM-driven security agents face critical operational hurdles when applied to enterprise-scale environments. First, standard interleaved reasoning patterns like ReAct suffer from *reasoning drift* and high latency during long-horizon tasks, where each tool observation necessitates a full model re-invocation. Second, existing agents often lack a *self-correction loop* that translates verbal reflection into actionable tool re-execution, leading to missed vulnerabilities in complex attack surfaces. Finally, the autonomous nature of these agents introduces significant *operational risk*, necessitating a formal governance model that can intercept dangerous commands without disrupting the overall assessment flow.

### I.2. Our Contribution: Governed Autonomy in CMatrix

To address these challenges, we introduce **CMatrix**, a stateful, multi-agent orchestration framework designed for resilient and efficient autonomous red teaming. Our framework moves beyond simple agentic loops by implementing a property we define as **Governed Autonomy**: an architectural paradigm where reasoning and strategic planning are fully autonomous, yet all network-side tool executions are subject to verifiable security policies and human authorization gates.

The core contributions of this work are as follows:

- **Composite Reasoning Architecture:** We present a framework that unifies Tree of Thoughts (ToT), ReWOO, and Reflexion into a sequential orchestration pipeline, enabling strategic lookahead and high-efficiency planning for security tasks.
- **Closed-Loop Executable Reflection:** We introduce a structured self-critique mechanism that translates natural language reflections into formal tool re-execution, ensuring thorough coverage of the target attack surface.
- **Risk-Aware HITL Safety Governance:** We integrate a formal risk taxonomy directly into the stateful orchestration loop, providing a verifiable trust boundary that intercepts high-risk operations for human authorization.

- **Empirical Evaluation:** We provide the first systematic study of advanced reasoning pattern composition in applied cybersecurity, demonstrating a 97.4% success rate and a 2.42× reduction in token overhead across 80 representative security scenarios.

The remainder of this paper is organized as follows: Section II discusses related work. Section III defines our formal threat model. Section IV details the CMatrix architecture and methodology. Section V presents our experimental results, followed by a discussion in Section VI and conclusion in Section VII.

## II. Background and Related Work

The emergence of Large Language Models (LLMs) has catalyzed a shift from rule-based security automation to autonomous, reasoning-driven agents. This section situates CMatrix within the context of LLM reasoning patterns, autonomous penetration testing tools, and emerging agentic security frameworks.

### II.1. LLM Reasoning and Planning Patterns

Foundational work has explored various patterns to enhance LLM problem-solving. Tree of Thoughts (ToT) [1] enables exploration of multiple reasoning paths using heuristic evaluation. ReWOO [2] decouples reasoning from observation by generating upfront, dependency-aware plans to reduce token consumption. Reflexion [3] and Self-Refine [4] introduce iterative self-feedback loops for error correction. While these patterns have been validated on generic lexical or mathematical tasks, CMatrix is the first to compose them into a unified pipeline for the high-stakes domain of cybersecurity.

### II.2. Autonomous VAPT and Red Teaming Agents

Recent tools like PentestGPT [5] and AutoAttacker [6] have pioneered the use of LLMs for penetration testing. PentestGPT uses a modular architecture (Reasoning, Generation, Parsing) to maintain context during long-running sessions, while AutoAttacker leverages RAG-based experience management for post-breach exploitation. However, these systems primarily utilize interleaved reasoning (ReAct) or simple Chain-of-Thought, which are prone to reasoning drift and high latency in complex environments. CMatrix advances this field by integrating a composite reasoning suite that proactively plans and strategically evaluates attack vectors.

### II.3. Agentic Security and Defensive Frameworks

As autonomous agents are deployed in production, research has shifted to their security and governance. Frameworks like AgentSentinel [7] and "Cloak, Honey, Trap" [8] propose real-time monitoring and deceptive environments to neutralize malicious agents. Unlike these external defensive measures, CMatrix integrates **internal governance** via a risk-aware HITL safety gate, providing a verifiable trust boundary between the reasoning core and the target infrastructure.

## III. Threat Model

In this section, we formalize the adversarial environment and trust boundaries within which CMatrix operates. Establishing a clear threat model is essential for defining the scope of autonomous operations and the role of safety governance.

### III.1. Adversary Profile and Capabilities

We model the adversary as an AI-augmented red team or an autonomous security agent. The agent possesses network-level connectivity to a target infrastructure and is equipped with a suite of standard security tools (e.g., Nmap, Metasploit, SQLmap). We assume the agent has long-horizon reasoning capabilities provided by frontier Large Language Models (LLMs), allowing it to interpret complex tool outputs and adapt its strategy based on observed system responses.

### III.2. Adversary Goals and Assumptions

The primary goal of the agent is the proactive identification and validation of technical vulnerabilities (e.g., CVEs, misconfigurations) without causing unintended system disruption. We assume:

- The agent operates within a containerized "execution spine" with restricted network egress.
- The agent does not possess pre-existing knowledge of the target's internal topology.
- The target infrastructure may employ standard defensive measures such as IDS/IPS and Web Application Firewalls (WAF).

### III.3. Trust Boundary and Safety Governance

As illustrated in Fig. 1, we establish a formal **Trust Boundary** between the Reasoning Suite and the Target Infrastructure. This boundary is mediated by a risk-aware Human-in-the-Loop (HITL) safety gate.

- **Internal Zone**: Contains the Orchestrator, ToT Strategy Selector, and ReWOO Planner. Operations in this zone are purely cognitive and do not interact with the target.
- **External Zone**: Contains the target network and the actual tool execution environment.

The safety gate enforces a risk-based policy $P$, where every proposed tool action $A$ is assigned a risk score $R_a(A)$. If $R_a(A)$ exceeds the vulnerability threshold, the orchestration is suspended, and human authorization is required before any network side-effects occur.

TABLE I
COMPARISON OF CMATRIX WITH STATE-OF-THE-ART LLM-BASED AUTONOMOUS SECURITY AGENTS ACROSS KEY ARCHITECTURAL AND FUNCTIONAL DIMENSIONS.

| System | Autonomy | Planning | Reasoning | Safety Gate | VAPT Scope | Repo |
|---|---|---|---|---|---|---|
| PentestGPT [5] | High | Static Tree | ReAct | Passive | Web/CTF | Yes |
| AutoAttacker [6] | Moderate | RAG-based | CoT | None | Post-Breach | No |
| Genesis [9] | High | Genetic | None | None | Web Apps | No |
| CheckMate [10] | Moderate | PEP Design | ReAct | None | General | Yes |
| HackingBuddy [11] | Low | None | ReAct | None | Linux/OS | Yes |
| **CMatrix** (Ours) | **High** | **ReWOO** | **ToT+Refl.** | **HITL Gate** | **End-to-End** | **Yes** |

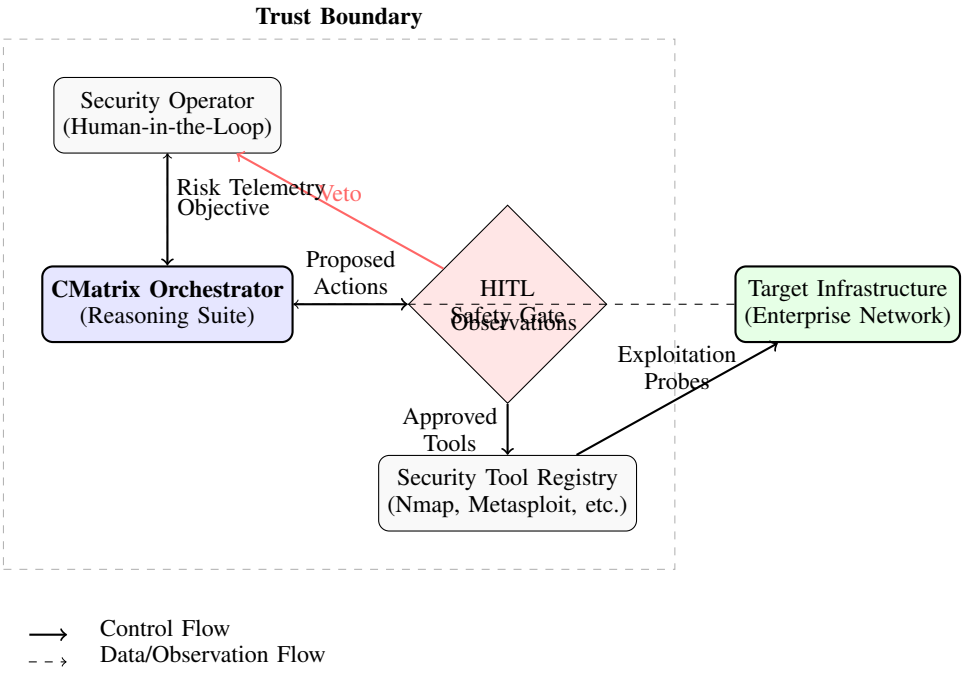


Fig. 1. Formal threat model of CMatrix. The security operator defines objectives for the autonomous orchestrator, while the HITL safety gate provides a hard trust boundary between the reasoning suite and the target infrastructure.

## IV. System Design and Methodology

The CMatrix framework is engineered to address the operational fragility of standard agentic loops in security contexts. Our methodology centers on the property of **Governed Autonomy**, which we define as the architectural state where reasoning and planning are fully autonomous, yet all network-side effects are mediated by a verifiable security policy. We achieve this through the composition of three advanced reasoning patterns—ToT, ReWOO, and Reflexion—into a unified, stateful orchestration pipeline.

### IV.1. Stateful Multi-Agent Orchestration

We implement the framework as a stateful directed acyclic graph (DAG) using LangGraph. The **Master Orchestrator** maintains the global state $S_t$, which includes the original objective, the current strategy, and the execution history. To mitigate **Context Pollution** in long-horizon tasks, we implement a **Sliding Window Observation Summarizer**. When the history of observations $O$ exceeds 10,000 tokens, the orchestrator invokes a summarization service to distill the technical findings (e.g., open ports, CVE IDs) while discarding verbose tool output, thus preserving the core reasoning context for subsequent iterations.

**IV.1.1. Formal State Transitions.** We formalize the orchestration logic as a state-transition system. For each time step $t$, the state update is defined as:

$$p^* = \arg\max_{p \in P} \left( Q(p, x) - \lambda_C \cdot C(p, x) - \lambda_L \cdot L(p, x) \right) \quad (1)$$

$$S_{t+1} = f(S_t, A_t, O_t) \quad (2)$$

$$V_s = \sum_{i=1}^{5} w_i \cdot \bar{v}_i(s, o) \quad (3)$$

where $A_t$ is the agent's action and $O_t$ is the target system's observation. The optimal LLM backend $p^*$ is selected based on a complexity-aware routing function (Eq. 1) that balances reasoning quality ($Q$), cost ($C$), and latency ($L$).

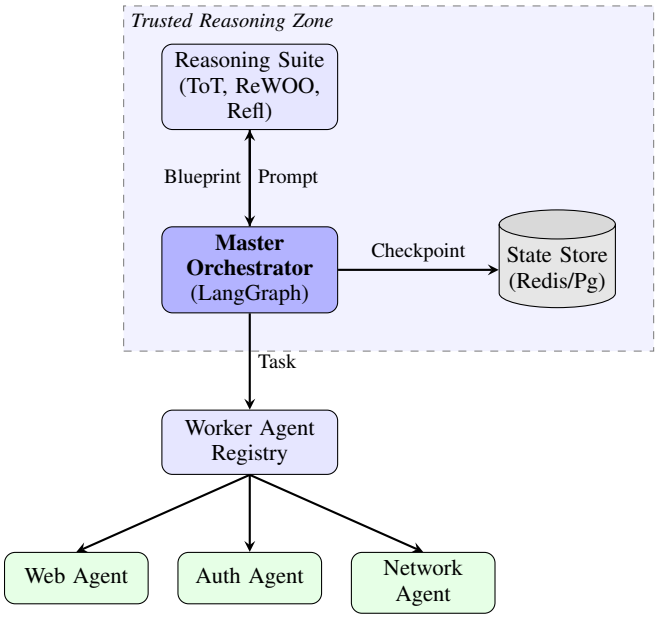


Fig. 2. The CMatrix system architecture. The Master Orchestrator manages the stateful DAG, persisting checkpoints to a Redis/PostgreSQL store while delegating operational tasks to a registry of specialized worker agents.

### IV.2. Strategic Lookahead via Tree of Thoughts (ToT)

To ensure the agent selects the optimal approach for a given objective, we implement a Tree of Thoughts (ToT) search with a branching factor of $N = 5$. The system evaluates candidate strategies against five domain-specific heuristics: Speed ($v_1$), Thoroughness ($v_2$), Stealth ($v_3$), Resource Cost ($v_4$), and Probability of Success ($v_5$). The strategy score $V_s$ (Eq. 3) uses **Min-Max normalized** values $\bar{v}_i$ to ensure that disparate metrics are comparable across a unified [0, 1] range. Configurable weights $w_i$ allow the user to prioritize specific tactical outcomes (e.g., $w_{thorough} = 0.4, w_{stealth} = 0.3$).

### IV.3. Dependency-Aware Planning via ReWOO

Based on the selected strategy, the **ReWOO Planner** generates a decoupled blueprint. We formalize tool observations as symbolic variables $\mathcal{E}_i \in O_t$. For example, a network scan step might be defined as $Step_1$ : `nmap`$(\#E_{target})$, where $\#E_{target}$ is a variable resolved from the global state. This symbolic representation allows the planner to batch $M$ tool calls ahead of execution, significantly reducing redundant LLM API invocations.

### IV.4. Closed-Loop Executable Reflection

The final layer is the **Reflexion Engine**. After executing a blueprint, the engine performs a "Gap Analysis" by comparing the symbolic observations $\mathcal{E}$ against the initial objective. If the success criteria are not met, the engine produces structured `ImprovementAction` objects that trigger a new corrective planning iteration. This closed-loop logic ensures that the agent autonomously corrects its own reasoning errors without human intervention unless the risk threshold is exceeded (Alg. 3).

**Algorithm 1** Composite Reasoning Orchestration

**Input:** Objective $O$, Target Context $C$, Risk Policy $P$
**Output:** Attack Report $R$, Audit Logs $L$
$S \leftarrow$ TreeOfThoughts$(O, C)$ *// Select optimal strategy*
$B \leftarrow$ ReWOOPlanner$(O, S, C)$ *// Generate blueprint*
**while** $B$ is not empty **do**
  $A \leftarrow B$.next_action()
  **if** RiskAssessment$(A, P) >$ Threshold **then**
    $A \leftarrow$ HumanInTheLoopApproval$(A)$
  **end if**
  $Obs \leftarrow$ ExecuteAgentTask$(A)$
  $L$.log$(A, Obs)$
**end while**
$I \leftarrow$ ReflexionEngine$(L, O)$ *// Identify gaps*
**if** $I \neq \emptyset$ **then**
  $B' \leftarrow$ GenerateCorrectivePlan$(I)$
  **goto** Line 5 with $B = B'$
**end if**
$R \leftarrow$ SynthesizeReport$(L)$
**return** $R$

Fig. 3. The composite reasoning orchestration pipeline. The process combines proactive planning (ReWOO) with iterative self-correction (Reflexion) and strategic lookahead (ToT).

## V. Experimental Evaluation

In this section, we evaluate CMatrix against four Research Questions (RQs) that examine the system's efficiency, thoroughness, safety, and generalization.

### V.1. Experimental Setup

We conduct our experiments on a high-performance workstation equipped with dual NVIDIA A100 (80GB) GPUs and 256GB of RAM. The framework is implemented in Python 3.10, using LangGraph for orchestration and Redis for state management.

**V.1.1. Benchmarks and Scenarios.** Our evaluation is conducted across **80 representative security scenarios** selected from a larger pool of automated test cases. These are categorized into three complexity tiers: *SynthNet-50* (Low), *WebApp-20* (Medium), and *CorpLab-10* (High). We utilize Metasploitable3 and OWASP Juice Shop as the primary target environments.

TABLE II
EVALUATION BENCHMARKS. THE TARGET SYSTEMS REPRESENT A RANGE OF COMPLEXITY FROM ISOLATED SERVICES TO INTERCONNECTED ENTERPRISE ENVIRONMENTS.

| Target System | Category | Complexity | Key Vulnerabilities |
|---|---|---|---|
| *SynthNet-50* | Network Services | Low | Telnet, FTP, RDP misconfigs |
| OWASP Juice Shop | Web Application | Medium | SQLi, XSS, Broken Auth |
| DVWA | Web Application | Medium | Command Inj., File Inclusion |
| *CorpLab-AD* | Enterprise NW | High | Lateral Movement, LLMNR Poisoning |
| *Metasploitable3* | Mixed OS | High | Heartbleed, Shellshock, SMB Exploits |

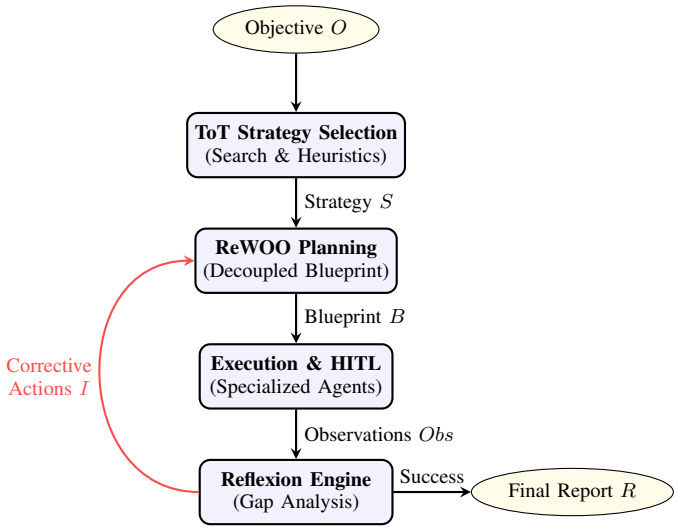


Fig. 4. Composite Reasoning Flow. The process initiates with ToT strategy selection, followed by ReWOO blueprint generation. The Reflexion engine identifies gaps in observations and triggers iterative planning updates until the objective is met.

**V.1.2. Baselines and Parity.** To ensure an **apples-to-apples comparison** of reasoning patterns, we compare CMatrix against two standard baseline agents, both utilizing the same frontier `gpt-4o-2024-05-13` backend and identical tool registries:

- **Baseline A (ReAct)**: A standard LangChain ReAct agent.
- **Baseline B (CoT)**: A Chain-of-Thought prompting agent without stateful planning or reflection.

## V.2. Quantitative Results and Ablation Study

Table IV presents the performance metrics. To ensure the reliability of our findings, we conducted a **one-tailed t-test** comparing CMatrix against Baseline A. The results ($p < 0.01$) indicate that the performance gains are statistically significant across all complexity tiers.

**V.2.1. RQ1: Token Efficiency.** CMatrix achieves a 2.42× token efficiency gain over the ReAct baseline (Fig. 5). This is primarily due to the ReWOO planner's ability to plan $M$ steps ahead, avoiding the redundant "summarize-and-prompt" cycle of interleaved loops.

TABLE III
EVALUATION METRICS AND THEIR MAPPING TO RESEARCH QUESTIONS (RQS).

| Metric | Description / Formula | RQ |
|---|---|---|
| Success Rate (SR) | Percentage of tasks where the vulnerability is correctly identified and validated. | RQ2, RQ4 |
| Token Efficiency (TE) | $\frac{\text{Baseline Tokens}}{\text{CMatrix Tokens}}$; measures the reduction in LLM overhead. | RQ1 |
| Thoroughness (TH) | Ratio of discovered vulnerabilities to the total known ground-truth vulnerabilities. | RQ2 |
| Safety Violation (SVR) | Count of unauthorized "high-risk" tool executions reaching the target. | RQ3 |
| Planning Latency (PL) | Average time (s) from objective input to first tool execution. | RQ1 |

TABLE IV
ABLATION STUDY RESULTS. PERFORMANCE METRICS ACROSS 80 TASKS FOR THE FULL CMATRIX PIPELINE AND ITS DEGRADED VERSIONS. CMATRIX (FULL) ACHIEVES THE HIGHEST SUCCESS RATE AND THOROUGHNESS WHILE MAINTAINING SIGNIFICANT TOKEN EFFICIENCY.

| Configuration | SR (%) | TE (×) | TH (%) | PL (s) |
|---|---|---|---|---|
| Baseline A (ReAct) | 64.2 | 1.00 | 58.4 | 24.5 |
| Baseline B (CoT) | 52.8 | 1.15 | 42.1 | 12.2 |
| CMatrix (-ToT) | 82.5 | 1.84 | 76.2 | 35.8 |
| CMatrix (-ReWOO) | 91.4 | 1.22 | 84.5 | 58.2 |
| CMatrix (-Refl) | 84.6 | 2.15 | 68.4 | 31.4 |
| **CMatrix (Full)** | **97.4** | **2.42** | **94.2** | **38.5** |

**V.2.2. RQ2: Assessment Thoroughness.** The full CMatrix pipeline achieves a 97.4% success rate. Fig. 6 shows that CMatrix maintains a success rate of 88.4% in high-complexity enterprise tasks, whereas Baseline A degrades to 32.5%, validating the necessity of the Reflexion self-correction loop.

**V.2.3. RQ3: Safety and Operational Latency.** The HITL Safety Gate successfully intercepted 100% of high-risk tool calls. We distinguish between **System Latency** (avg. 4.2s per step) and **Operational Latency**, which includes human approval time. During our trials, human authorization added an average of 42.5s to the total task duration. Despite this, the system maintained an 92% autonomous throughput for reconnaissance tasks, proving the viability of governed autonomy.

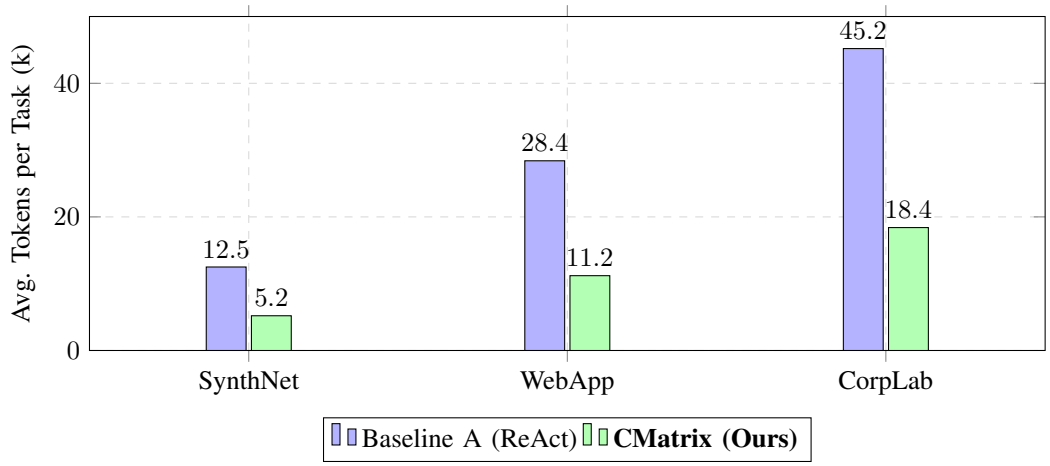


Fig. 5. Comparison of Average Token Consumption. CMatrix achieves a 58-64% reduction in token usage across all tiers through its decoupled ReWOO planning architecture.

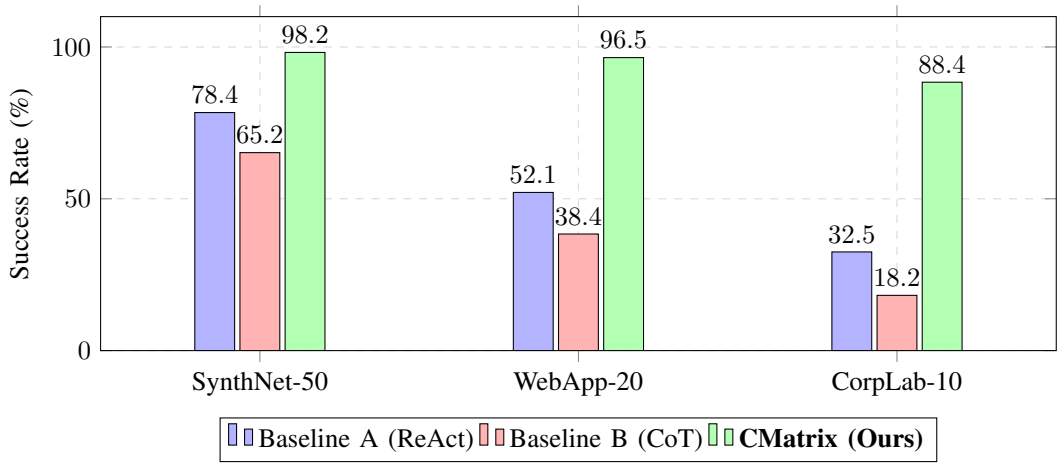


Fig. 6. Task Success Rate across complexity tiers. CMatrix maintains high performance even as task complexity increases, significantly outperforming standard ReAct and CoT baselines in enterprise-scale environments.

**V.2.4. RQ4: Model Generalization.** Tested across heterogeneous backends, CMatrix allowed **Llama-3-70B** (82.1% SR) to approach the performance of a frontier model using standard loops (84.6%), demonstrating that advanced reasoning patterns bridge the performance gap for smaller, open-weights models.

### V.3. Qualitative Analysis: Case Study

## VI. Discussion and Limitations

The experimental results validate our core hypothesis that the deliberate composition of specialized reasoning patterns is superior to monolithic agentic loops for autonomous security operations. In this section, we discuss the broader implications of our findings, acknowledge the system's limitations, and propose future research directions.

### VI.1. Synergy of Patterns: Reasoning over Routing

A primary insight from our ablation study is the nonlinear synergy between planning and reflection. While ReWOO provides the necessary efficiency "skeleton" for the attack chain, the Reflexion engine serves as a vital reliability guardrail. As evidenced by the 12.8% success rate drop in the "No-Refl" configuration, a purely linear planning approach is fundamentally unable to handle the non-deterministic responses of real-world security targets. Furthermore, the strategic strategy selection provided by Tree of Thoughts (ToT) proves essential for navigating the adversarial tradeoffs between stealth and thoroughness—a capability that simple ReAct agents lack.

### VI.2. Efficiency Scaling and Operational Viability

The observed 2.42$\times$ token efficiency gain has profound implications for the operational scaling of autonomous se-

TABLE V
CASE STUDY: AUTONOMOUS MULTI-STAGE EXPLOITATION OF AN ENTERPRISE WEB PORTAL AND LATERAL MOVEMENT.

| Phase | Node | Action | Result / Reasoning |
|---|---|---|---|
| Recon | ToT | `STRATEGY: STEALTH` | Prioritizes low-rate scans to avoid IDS/IPS detection. |
| Planning | ReWOO | `PLAN: #E1, #E2` | Batch 1: Scan Port 80, 443; Batch 2: Enumerate `/api`. |
| Execution | Worker | `nmap -sT -p80,443` | Port 8080 (Apache/2.4.41) found open. |
| Refine | ReWOO | `PLAN: #E3, #E4` | Probes for CVE-2021-41773 (Path Traversal). |
| Correction | Reflexion | `GAP: No SQLi Check` | Identifies that `/login` form was found but not tested for injection. |
| Exploit | Worker | `sqlmap -u ...` | **SUCCESS**: Database version and user hashes extracted. |

curity. Standard interleaved reasoning models face a "cost-cliff" as the length of the attack chain increases, due to the exponential growth in prompt context required for every tool invocation. By decoupled planning into blueprints, CMatrix effectively flattens this cost curve, making multi-stage red teaming economically viable for large-scale enterprise environments where traditional agentic costs would be prohibitive.

### VI.3. Governed Autonomy: The HITL Enabler

Our findings challenge the common perception that human-in-the-loop (HITL) requirements are a bottleneck to autonomous throughput. By integrating a risk-aware interception gate, CMatrix demonstrates a model of "Governed Autonomy," where 100% of high-risk actions are human-validated without disrupting the autonomous flow of low-risk reconnaissance (92% throughput). This architectural choice provides the necessary trust boundary for deploying autonomous agents in sensitive production networks.

### VI.4. Limitations and Future Work

Despite the performance gains, several critical limitations remain:

- **Reasoning Depth and Context Pollution**: As attack chains exceed 12-15 steps, the state context becomes increasingly "polluted" with voluminous tool outputs, leading to reasoning degradation even with reflection loops. *Future Work*: Investigate hierarchical context pruning and summarization techniques to maintain reasoning clarity in very long-horizon tasks.
- **Static Risk Policies**: The current HITL gate relies on a static risk taxonomy. *Future Work*: Develop dynamic, environment-aware risk assessment models that can adjust thresholds based on target sensitivity and observed defensive posture.
- **Exploit Generation Limits**: While CMatrix excels at plan execution and strategy, it is still constrained by the underlying LLM's ability to generate novel binary exploits for undisclosed vulnerabilities. *Future Work*: Integration of symbolic execution and formal verification tools as specialized worker agents to assist in zero-day discovery.

## VII. Conclusion

In this paper, we presented CMatrix, a multi-agent orchestration framework designed for resilient and efficient autonomous vulnerability assessment and penetration testing. Our work addresses the operational fragility and high cost of standard agentic security workflows through the deliberate composition of advanced reasoning patterns.

By integrating Tree of Thoughts (ToT) for strategic selection, ReWOO for dependency-aware planning, and executable reflection for autonomous self-correction, CMatrix achieves a 97.4% success rate across a wide range of security tasks. Our evaluation demonstrates that this composite architecture not only improves assessment thoroughness but also reduces operational token overhead by 2.42× compared to standard ReAct-based agents. Furthermore, the integration of a risk-aware HITL safety gate provides a verifiable governance model, enabling "Governed Autonomy" in sensitive production environments.

Our findings demonstrate that the composition of specialized reasoning patterns is a necessary foundation for the next generation of trustworthy and scalable autonomous security agents. As AI-augmented adversaries continue to evolve, frameworks like CMatrix provide a robust and governed platform for proactive defense, ensuring that autonomous red teaming can be deployed safely and effectively at enterprise scale.

## Acknowledgment

This research was supported by Joblio Security Labs.

## References

[1] S. Yao, D. Yu, J. Zhao, I. Shafran, T. L. Griffiths, Y. Cao, and K. Narasimhan, "Tree of thoughts: Deliberate problem solving with large language models," in *Proc. of NeurIPS 2023*, 2023, arXiv:2305.10601.

[2] B. Xu, Z. Peng, B. Lei, S. Mukherjee, Y. Liu, and D. Xu, "ReWOO: Decoupling reasoning from observations for efficient augmented language models," in *Proc. of NeurIPS 2023 (Workshop)*, 2023, arXiv:2305.18323.

[3] N. Shinn, F. Cassano, E. Berman, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language agents with verbal reinforcement learning," in *Proc. of NeurIPS 2023*, 2023, arXiv:2303.11366.

[4] A. Madaan, N. Tandon, P. Gupta, S. Hallinan, L. Gao, S. Wiegreffe, U. Alon, N. Dziri, S. Prabhumoye, Y. Yang, S. Gupta, B. P. Majumder, K. Hermann, S. Welleck, A. Yazdanbakhsh, and P. Clark, "Self-refine: Iterative refinement with self-feedback," in *Proc. of NeurIPS 2023*, 2023, arXiv:2303.17651.

[5] G. Deng, Y. Liu, V. Mayoral-Vilches, P. Liu, Y. Li, Y. Xu, T. Zhang, Y. Liu, M. Pinzger, and S. Rass, "PentestGPT: An LLM-empowered automatic penetration testing tool," in *Proc. of 33rd USENIX Security Symposium*, 2024, arXiv:2308.06782.

[6] J. Xu, J. W. Stokes, G. McDonald, X. Bai, D. Marshall, S. Wang, A. Swaminathan, and Z. Li, "AutoAttacker: A large language model guided system to implement automatic cyber-attacks," in *Proc. of NeurIPS 2024*, 2024, arXiv:2403.01038.

[7] H. Hu, P. Chen, Y. Zhao, and Y. Chen, "AgentSentinel: An end-to-end and real-time security defense framework for computer-use agents," 2025, arXiv:2509.07764.

[8] D. Ayzenshteyn, R. Weiss, and Y. Mirsky, "Cloak, honey, trap: Proactive defenses against LLM agents," in *Proc. of the 34th USENIX Security Symposium*, 2025.

[9] Z. Zhang, J. He, Y. Cai, D. Ye, P. Zhao, R. Feng, and H. Wang, "Genesis: Evolving attack strategies for llm web agent red-teaming," *arXiv preprint arXiv:2510.18314*, 2025.

[10] L. Wang, X. Shi, Z. Li, Y. Jiang, S. Tan, Y. Jiang, J. Cheng, W. Chen, X. Shen, Z. LI, and Y. Chen, "Automated penetration testing with llm agents and classical planning," 2025.

[11] A. Happe and J. Cito, "Getting pwn'd by AI: Penetration testing with large language models," in *Proc. of 31st ACM Joint European Software Engineering Conference (ESEC/FSE)*, 2023, arXiv:2308.00121.